# Lab 22 — Site-to-Site VPN with WireGuard

> **SY0-701 Objective 3.2** — Given a scenario, apply security principles to secure enterprise infrastructure
>
> **Domain 3** — Security Architecture (18%) · **Time:** ~25 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

**Goal:** Build an encrypted site-to-site tunnel between two simulated sites, prove on the wire that the traffic is encrypted, and show how AllowedIPs restricts what may enter the tunnel.

**You'll build:** Two network namespaces (Site A and Site B) joined by a WireGuard tunnel, a packet capture showing ciphertext only, and a crypto-key routing test.

**Prerequisites:** Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 19 (namespaces) is helpful; tools installed in step 1.

## Background

- A **VPN** builds an encrypted tunnel across an untrusted network, protecting the confidentiality and integrity of **data in transit**. A **site-to-site** VPN joins two networks through gateways; **remote access** VPN connects one user device.
- **Tunnelling** choices on the exam: **IPSec** (layer 3; AH for integrity, ESP for encryption; IKE on UDP 500/4500; tunnel or transport mode) and **TLS** VPNs over TCP 443. WireGuard is a modern layer 3 tunnel with fixed cryptography (Curve25519, ChaCha20-Poly1305).
- WireGuard peers authenticate each other with **public keys**, like SSH authorised keys. **AllowedIPs** is both a routing table and an access list: a peer may only send from, and receive for, the addresses listed for it (crypto-key routing).
- **Split tunnel** sends only corporate subnets through the VPN; **full tunnel** sends everything. **SD-WAN** and **SASE** build on the same idea at enterprise scale, steering encrypted branch traffic by policy and delivering security from the cloud.

## Steps

### 1. Install WireGuard and capture tools

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y wireguard-tools
iproute2 iputils-ping kmod tcpdump netcat-openbsd
modprobe wireguard && lsmod | grep wireguard
```

### 2. Generate a key pair for each site

Private keys are created with a restrictive umask so only root can read them.

```
mkdir -p /root/wg && cd /root/wg && umask 077
for s in siteA siteB; do wg genkey | tee $s.key | wg pubkey > $s.pub; done
```

```
ls -l /root/wg
echo "Site A public key: $(cat siteA.pub)"
```

### 3. Create two site namespaces with WireGuard links

The interfaces are created in the host and then moved, so their encrypted UDP sockets stay on the host loopback — the simulated internet between the sites.

```
ip netns add siteA
ip netns add siteB
ip link add wgA type wireguard
ip link add wgB type wireguard
ip link set wgA netns siteA
ip link set wgB netns siteB
ip netns exec siteA ip link show wgA
```

### 4. Configure keys, listen ports and allowed IPs

Each site gets its private key, a listen port, its peer's public key and the single tunnel address that peer may use.

```
cd /root/wg
ip netns exec siteA wg set wgA private-key siteA.key listen-port 51820 \
  peer "$(cat siteB.pub)" allowed-ips 10.99.0.2/32
ip netns exec siteB wg set wgB private-key siteB.key listen-port 51821 \
  peer "$(cat siteA.pub)" allowed-ips 10.99.0.1/32
ip netns exec siteA ip addr add 10.99.0.1/24 dev wgA
ip netns exec siteB ip addr add 10.99.0.2/24 dev wgB
```

### 5. Point each peer at the other's endpoint

The endpoint is the public address of the other gateway; here both gateways share the host loopback.

```
cd /root/wg
ip netns exec siteA wg set wgA peer "$(cat siteB.pub)" endpoint 127.0.0.1:51821
ip netns exec siteB wg set wgB peer "$(cat siteA.pub)" endpoint 127.0.0.1:51820
ip netns exec siteA ip link set wgA up
ip netns exec siteB ip link set wgB up
```

### 6. Verify the tunnel with ping and wg show

A successful ping triggers the handshake; `wg show` then reports it and counts the encrypted bytes.

```
ip netns exec siteA ping -c3 10.99.0.2
ip netns exec siteA wg show
```

```
interface: wgA
  public key: ...
  listening port: 51820

peer: ...
  endpoint: 127.0.0.1:51821
```

```
  allowed ips: 10.99.0.2/32
  latest handshake: 2 seconds ago
  transfer: 732 B received, 828 B sent
```

### 7. Capture the underlay to prove the traffic is encrypted

Send a "secret" message through the tunnel while capturing the UDP packets that actually cross the simulated internet.

```
cd /root/wg
timeout 12 tcpdump -i lo -nn -A -c 20 'udp port 51820 or udp port 51821' > capture.txt
2>/dev/null &
sleep 2
ip netns exec siteB sh -c 'timeout 6 nc -l 10.99.0.2 9000 > /root/wg/received.txt' &
sleep 1
echo "PAYROLL-TOP-SECRET-2024" | ip netns exec siteA nc -q 1 -w 3 10.99.0.2 9000
wait
echo "Received at Site B: $(cat received.txt)"
echo "Encrypted packets captured: $(grep -c 'UDP' capture.txt)"
grep -q "PAYROLL-TOP-SECRET" capture.txt && echo "PLAINTEXT LEAKED" || echo "Plaintext not
visible on the wire - encrypted"
```

### 8. Test crypto-key routing with AllowedIPs

Give Site B a second address that is not in Site A's AllowedIPs list. Site A has no peer key for it, so the packet is refused before it is ever encrypted.

```
ip netns exec siteB ip addr add 10.99.0.3/24 dev wgB
ip netns exec siteA ping -c2 -W1 10.99.0.3 2>&1 | grep -E 'Required key|Unreachable|packet
loss' | head -2
ip netns exec siteA wg show wgA allowed-ips
```

```
ping: sendmsg: Required key not available
2 packets transmitted, 0 received, 100% packet loss
```

Depending on the kernel you see `Required key not available` or simply 100% packet loss — either way WireGuard refuses to send traffic for an address that no peer is authorised to use.

## Test it

```
ip netns exec siteA ping -c2 -W2 10.99.0.2 | grep -E '[0-9]+ received'
ip netns exec siteA wg show wgA latest-handshakes
grep -c "PAYROLL-TOP-SECRET" /root/wg/capture.txt
```

Expected: `2 received`, a non-zero Unix timestamp for the latest handshake, and `0` (the secret never appears in the capture).

## Troubleshooting

| Symptom | Fix |
| --- | --- |
| `modprobe: FATAL: Module wireguard not found` | The kernel is older than 5.6 without the module; install `wireguard-dkms` or use a newer Ubuntu playground. |
| `ping` fails and `wg show` has no handshake | Check both endpoints (step 5) and that each `allowed-ips` holds the other site's address. |
| `RTNETLINK answers: File exists` | The lab was run before; continue from step 6 or start a fresh playground. |

## Challenge

Turn this into a full site-to-site design: add a LAN subnet behind each gateway (for example 192.168.10.0/24 in Site A and 192.168.20.0/24 in Site B on dummy interfaces), extend each peer's AllowedIPs to include the remote LAN, add routes, and prove LAN-to-LAN traffic flows only through the tunnel.

## Reflection

Your company is replacing its MPLS links with SD-WAN over broadband. Which part of this lab protects the branch traffic, and what does AllowedIPs contribute that a plain encrypted tunnel would not?

## Exam tips

- IPSec AH = integrity and authentication only; ESP = encryption plus integrity. Tunnel mode wraps the whole packet (gateways); transport mode protects the payload (host to host).
- TLS VPNs run over TCP 443 and pass most firewalls; IPSec uses UDP 500/4500 and ESP (IP protocol 50).
- Split tunnel saves bandwidth but internet traffic skips corporate inspection; full tunnel inspects everything.
- SASE = cloud-delivered security (SWG, CASB, ZTNA, FWaaS) plus networking; SD-WAN = policy-based WAN routing over mixed links.

## References


- WireGuard whitepaper and quick start — https://www.wireguard.com/
- WireGuard network namespace integration — https://www.wireguard.com/netns/
- NIST SP 800-77 Rev. 1 Guide to IPsec VPNs — https://csrc.nist.gov/pubs/sp/800/77/r1/final
- NIST SP 800-113 Guide to SSL VPNs — https://csrc.nist.gov/pubs/sp/800/113/final


---